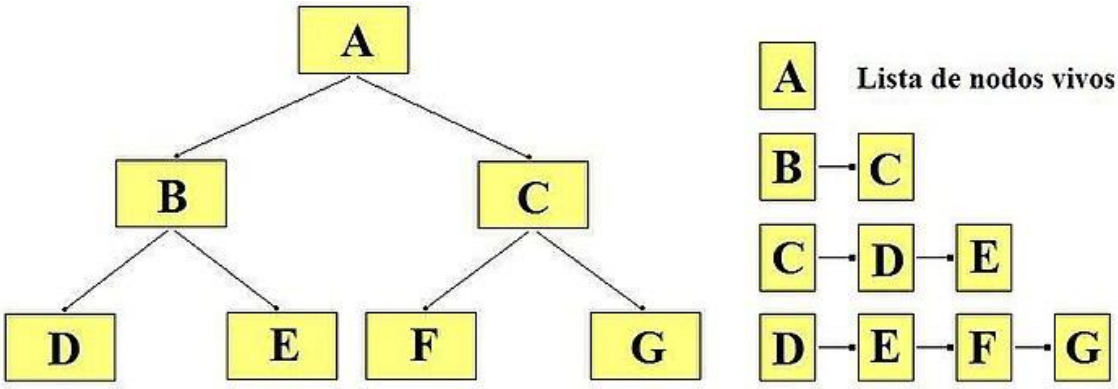
PRÁCTICA FINAL – *Ramificación y Poda*

Análisis y Diseño de Algoritmos

UNIVERSIDAD DE ALICANTE



Aitana Cantó Rico

ÍNDICE

1. Estructuras de datos 2
1.1 Nodo 3
1.2 Lista de nodos vivos 3
2. Mecanismos de poda 3
2.1. Poda de nodos no factibles 3
2.2. Poda de nodos no prometedores 4
3. Cotas pesimistas y optimistas 4
3.1 Cota pesimista inicial (inicialización). 4
3.2 Cota pesimista del resto de nodos. 5
3.3 Cota optimista. 6
4. Otros medios empleados para acelerar la búsqueda 6
5. Estudio comparativo de distintas estrategias de búsqueda 7
6. Tiempos de ejecución. 7

1. Estructuras de datos

- **MapCoord**: Estructura que contiene la matriz de dificultades del mapa y sus dimensiones.

```
// Struct del mapa de coordenadas
struct MapCoord {
    vector<vector<int>> weight;
    int column;
    int row;
};
```

- **Data**: Contiene diversas estructuras auxiliares para almacenar los mejores caminos, los costes asociados y la cola de prioridad para la ramificación.

```
struct Data {
    vector<vector<long>> almacenTopSolu; // Almacén de mejor coste para cada celda
    vector<vector<bool>> noIter; // Matriz para diferenciar si la solución proviene del Iterativo
    long bestSolu;    // Mejor solución encontrada
    vector<int> movementsBestSolu; // Movimientos mejor solución
    priority_queue<Nodo, vector<Nodo>, greater<Nodo>> cola; // Cola Prioridad
    vector<vector<int>> pessimisticPath; //Camino de la cota pesimista
    vector<vector<int>> path2Cell; //Camino desde el inicio del mapa a la celda
    vector<vector<long>> matrizPesimista; // Matriz de la cota pesimista
};
```

- **Stadistics**: Estructura para registrar estadísticas de la ejecución del algoritmo, como nodos visitados, nodos explorados, y actualizaciones de la mejor solución.

```
struct Stadistics{
    long visited_nodes = 0;          // Nodos Visitados
    long explored_nodes = 0;      // Nodos explorados
    long visited_leaf_nodes = 0; // Nodos Hoja Visitados
    long no_promissing_discarded_nodes = 0; // Nodos No prometedores
    long no_feasible_discarded_nodes = 0;     // Nodos No factibles
    long npromising_but_discarded = 0;       // Nodos prometedores descartados
    long nbest_solution_updated_from_leafs = 0;
    long nbest_solution_updated_from_pessimistic_bound = 0;
};
```

1.1 Nodo

En este apartado se describirán brevemente las variables que componen el nodo y su cometido. Un nodo se define como una tupla que contiene cuatro elementos:

```
using Nodo = tuple<float, long, long, long>;
```

- **float:** Representa la cota optimista estimada del nodo.
- **long:** El coste acumulado hasta el nodo actual.
- **long:** La fila en la que se encuentra el nodo.
- **long:** La columna en la que se encuentra el nodo.

1.2 Lista de nodos vivos

La lista de nodos vivos se gestiona mediante una cola de prioridad (`priority_queue`), tal y como se explica en las diapositivas de la asignatura. Esta estructura permite la extracción eficiente del nodo más prometedor, que es el nodo con la menor cota. Esta pertenece al Struct Data:

```
priority_queue<Nodo, vector<Nodo>, greater<Nodo>> cola; // Cola Prioridad
```

Además, se inicializa con un comparador `greater<Nodo>`, lo que garantiza que el nodo con la menor cota siempre esté en la cima de la cola y sea extraído primero. Otros criterios de extracción, como el orden de inserción o la cota optimista, se consideran menos eficientes para este problema y no se han implementado.

2. Mecanismos de poda

2.1. Poda de nodos no factibles

Los nodos no factibles se descartan mediante una verificación de los límites del mapa. Si un nodo se encuentra fuera de los límites del mapa, se considera no factible y se descarta.

```
if(rDesp<=lastRow && rDesp>=0 && cDesp<=lastCol && cDesp>=0){ // Solucion Factible
  // Nodo Factible
}
else{
        s.no_feasible_discarded_nodes++;
}
```

2.2. Poda de nodos no prometedores

Los nodos no prometedores se descartan comparando la suma de su cota optimista y su costo acumulado con la mejor solución encontrada hasta el momento (`d.bestSolu`). Si esta suma es mayor o igual que la mejor solución, el nodo se descarta.

```
if(cotaOptimista + sol < d.bestSolu){ // Nodo Prometedor  }
else{   s.no_promissing_discarded_nodes++;    }
```

3. Cotas pesimistas y optimistas

3.1 Cota pesimista inicial (inicialización).

La cota pesimista inicial se calcula utilizando la función `mcp_it`, implementada en la práctica 6, que determina el coste mínimo desde la celda inicial hasta la celda final, teniendo en cuenta el peso de cada celda y las posibles direcciones de movimiento (arriba, izquierda, diagonal).

```
long mcp_it(const MapCoord &mapC, vector<vector<long>> &aux) {
    // CREAMOS VECTOR y lo inicializamos a 0
    aux = vector<vector<long>>(mapC.row, vector<long>(mapC.column, 0));
    // 1a FILA
    for (int j = 0; j < mapC.column; j++) {
        if (j == 0) {   aux[0][j] = mapC.weight[0][j];  }
        else {          aux[0][j] = mapC.weight[0][j] + aux[0][j - 1];  }
    }
    // 1a COLUMNA
    for (int i = 1; i < mapC.row; i++) {
        aux[i][0] = mapC.weight[i][0] + aux[i - 1][0];
    }
    // RESTO MATRIX
    for (int i = 1; i < mapC.row; i++) {
        for (int j = 1; j < mapC.column; j++) {
            long arriba = aux[i - 1][j];
            long izquierda = aux[i][j - 1];
            long diagonal = aux[i - 1][j - 1];
            aux[i][j] = mapC.weight[i][j] + smallest(arriba, izquierda, diagonal);
        }
    }
    return aux[mapC.row - 1][mapC.column - 1];
}
```

El valor devuelto de esta función es nuestra cota pesimista inicial que se instancia en el main:

```
d.bestSolu = mcp_it(mapC, d.almacenTopSolu);
```

De esta función también extraemos *almacenTopSolu* que es una matriz que contine el mejor coste calculado para cada una de las celdas del mapa.

3.2 Cota pesimista del resto de nodos.

Para calcular la cota pesimista se ha implementado la función *cota_pesimista*. Esta función calcula una matriz de costes pesimistas para llegar a cada una de las celdas desde la esquina inferior derecha. Además, genera un camino pesimista que indica la dirección a seguir para alcanzar dicho coste en cada celda.

```
vector<vector<long>> cota_pesimista(vector<vector<int>> &pessimisticPath, MapCoord m) {
    int lastRow = m.row - 1;
    int lastCol = m.column - 1;
    long r1, r2, r3;

    vector<vector<long>> matrizPesimista;      // Inicializar matrizPesimista a 0
    for (int i = 0; i < m.row; ++i) {
        vector<long> row;
        for (int j = 0; j < m.column; ++j) {     row.push_back(0);       }
        matrizPesimista.push_back(row);
    }
    for (int i = 0; i < m.row; ++i) {    // Inicializar pessimisticPath a 0
        vector<int> row;
        for (int j = 0; j < m.column; ++j) { row.push_back(0);      }
        pessimisticPath.push_back(row);
    }
    // Inicializar la esquina inferior derecha
    pessimisticPath[lastRow][lastCol] = 0;
    matrizPesimista[lastRow][lastCol] = m.weight[lastRow][lastCol];
    // Inicializar última fila y última columna
    for (int r = lastRow - 1; r >= 0; r--) {
        pessimisticPath[r][lastCol] = 5;
        matrizPesimista[r][lastCol] = matrizPesimista[r + 1][lastCol] + m.weight[r][lastCol];
    }
    for (int c = lastCol - 1; c >= 0; c--) {
        pessimisticPath[lastRow][c] = 3;
        matrizPesimista[lastRow][c] = matrizPesimista[lastRow][c + 1] + m.weight[lastRow][c];
    }
    for (int r = lastRow - 1; r >= 0; r--) {    // Calcular valores para el resto de la matriz
        for (int c = lastCol - 1; c >= 0; c--) {
            r1 = matrizPesimista[r + 1][c + 1];
            r2 = matrizPesimista[r][c + 1];
            r3 = matrizPesimista[r + 1][c];
            matrizPesimista[r][c] = m.weight[r][c] + smallest(r1, r2, r3);

            if (r3 == smallest(r3, r2, r1)) {    pessimisticPath[r][c] = 5;}
            else if (r2 == smallest(r3, r2, r1)) { pessimisticPath[r][c] = 3; }
           else {    pessimisticPath[r][c] = 4;  }
        }
    }
    return matrizPesimista;
}
```

3.3 Cota optimista.

La cota optimista se calcula como el coste mínimo estimado necesario para alcanzar la celda final desde la celda actual. Esto se basa en el menor elemento del mapa y la distancia restante a la celda final.

```
long cotaOptimista = 0;
if(rDesp > cDesp){  cotaOptimista = (lastCol - cDesp)* smallest; }
else{               cotaOptimista = (lastRow - rDesp)* smallest; }
```

Cálculo *smallest*:

```
    //Calculamos el menor elemento del mapa
    long smallest = infinito;
    for(int i = 0; i < m.row; i++){
        for(int j = 0; j<  m.column; j++){
            if(m.weight[i][j] < smallest){
                smallest = m.weight[i][j];
            }
        }
    }
```

"infinito" es una variable global declarada al principio del programa:

```
const long infinito = numeric_limits<long>::max();
```

4. Otros medios empleados para acelerar la búsqueda

En esta sección se describen las técnicas adicionales empleadas para mejorar la eficiencia del algoritmo. Estas técnicas complementan las estrategias de poda y las cotas optimistas y pesimistas, proporcionando un enfoque más robusto para reducir el tiempo de ejecución.

En mi caso, una mejora que he implementado es la utilización de la matriz de control `noIter` de tipo `vector<vector<bool>>` añadida al *struct Data*. Su función es distinguir si una solución proviene del algoritmo iterativo. Esta matriz permite evitar cálculos redundantes, evita la recalculación innecesaria de soluciones para celdas que ya tienen una solución óptima conocida, acelerando el proceso de búsqueda, concluyendo en una mejora significativa en el tiempo de ejecución del algoritmo.

```
long calculated = sol + m.weight[rDesp][cDesp];
long stored = d.almacenTopSolu[rDesp][cDesp];
bool difIterREcur = (!d.noIter[rDesp][cDesp] || (d.noIter[rDesp][cDesp] && (calculated < stored)));
if(difIterREcur){
   d.noIter[rDesp][cDesp] = true;
   d.almacenTopSolu[rDesp][cDesp] = calculated;
}
```

Otro medio implementado es el uso de `greater` en la Cola de Prioridad. Este es usado para optimizar la selección del siguiente nodo a procesar y permite extraer el nodo más prometedor de manera eficiente, priorizando aquellos nodos con menor coste estimado.

```
priority_queue<Nodo, vector<Nodo>, greater<Nodo>> cola; // Cola Prioridad
```

5. Estudio comparativo de distintas estrategias de búsqueda

A continuación, una tabla de tiempos donde se puede apreciar la diferencia tiempos en diferentes mapas de utilizar la matriz `noIter` y no utilizarla:

Fichero de test	*Tiempo (ms) usando noIter*	*Tiempo (ms) sin noIter*	*% retardo*
060.map	3,384	3,812	12,65%
090.map	6,387	7,123	11,52%
201.map	34,316	37,821	10,21%
301.map	69,912	77,492	10,84%
501.map	159,027	176,233	10,82%
700.map	306,063	337,321	10,21%
900.map	501,934	555,465	10,66%
1K.map	650,883	718,416	10,38%
2K.map	2859,83	3.153,825	10,28%
3K.map	6795,03	7.510,246	10,53%
4K.map	12631,7	13.922,231	10,22%
		% medio:	10,76%

Como se puede observar, no utilizar la matriz incrementa el tiempo de ejecución en un 10% aproximadamente.

6. Tiempos de ejecución.

Los tiempos de ejecución se miden utilizando la función `clock()` y se reportan en milisegundos.

```
auto start = clock();
  d.matrizPesimista = cota_pesimista(d.pessimisticPath, mapC);
  d.bestSolu = mcp_it(mapC, d.almacenTopSolu);
  mcp_bb(d, mapC, stadistics);
auto end = clock();

cout << 1000*(double)(end-start)/(double)CLOCKS_PER_SEC << endl;
```

Como se puede ver en el código anterior, para el cálculo del tiempo de ejecución se tiene en cuenta tanto la ejecución del algoritmo, como el del cálculo de la cota pesimista y también la cota inicial con el algoritmo iterativo. A continuación, los tiempos obtenidos para los ficheros test correspondientes:

Fichero de test	***Tiempo (ms)***
060.map	3.384
090.map	6.387
201.map	34.316
301.map	69.912
501.map	159.027
700.map	306.063
900.map	501.934
1K.map	650.883
2K.map	2859.83
3K.map	6795.03
4K.map	12631.7

Nota*: El ordenador en el cual se han ejecutados los mapas de prueba posee las siguientes especificaciones:*

- Procesesador: 12th Gen Intel(R) Core (TM) i7-1255U 1.70 GHz
- RAM: 16,0 GB (15,7 GB usable)
- Tipo de sistema: Sistema operativo de 64 bits, procesador basado en x64